

Objectif

Ce document définit les règles de conception de l'interface utilisateur pour l'application.

Il complète **AGENTS.md** en précisant les choix ergonomiques, visuels et structurels attendus pour l'IHM.

L'objectif est de produire une interface :

- cohérente,
- modulaire,
- lisible,
- orientée production,
- adaptée à la manipulation de blocs, médias, références, conteneurs et vues de travail.

Cette IHM doit évoquer un outil créatif professionnel moderne, sans sacrifier la simplicité d'usage ni la maintenabilité du code.

Philosophie générale de l'interface

L'application doit être pensée comme un **atelier visuel de travail**, pas comme un simple formulaire de gestion.

L'utilisateur manipule :

- des blocs,
- des conteneurs,
- des médias,
- des références,
- des relations, et des graphes
- des vues de navigation,
- des vues de détail,
- des panneaux de propriétés,
- des prévisualisations.

L'interface doit donc privilégier :

- la clarté structurelle,
 - la visibilité permanente des éléments utiles,
 - la réduction de la charge cognitive,
 - l'accès rapide aux objets,
 - la stabilité visuelle,
 - la densité utile sans surcharge.
-

Principes ergonomiques majeurs

1. Navigation à gauche, travail au centre, inspection à droite

Structure générale par défaut :

- **haut*** : toolbar avec toolbutton et / ou toggle togglebuttons pour afficher / masquer des panels
- **gauche** : navigation, arbre, filtres, collections, regroupements
- **centre** : vue principale de travail
- **droite** : inspecteur, propriétés, métadonnées, liens, paramètres contextuels

C'est le layout de référence pour la plupart des écrans.

2. Toujours distinguer navigation, contenu et propriétés

Ne pas mélanger dans un même panneau :

- l'arborescence,
- la zone de travail,
- les propriétés détaillées.

Chaque zone doit avoir un rôle clair.

3. La vue principale montre le travail, pas l'administration

Le centre doit servir en priorité à :

- visualiser,
- composer,
- comparer,
- sélectionner,
- ouvrir,
- organiser visuellement.

Éviter d'en faire une simple liste technique de champs.

4. Les propriétés détaillées sont secondaires mais toujours accessibles

Les propriétés ne doivent pas encombrer la zone principale.

Elles doivent rester disponibles dans un inspecteur stable à droite, En utilisant par exemple dans la zone des zone repliables

5. L'interface doit être robuste aux écrans complexes

Même si le domaine devient riche, l'utilisateur doit comprendre rapidement :

- où il est,
- ce qu'il regarde,
- ce qu'il peut faire,
- ce qui est sélectionné,
- ce qui est relié,
- ce qui est éditable.

Structure globale de la fenêtre principale

Main window

La fenêtre principale doit être pensée comme un shell d'application.

Elle contient généralement :

- barre de menu (pour les grandes fonctions)
- barre d'outils principale
- zone centrale de travail
- panneaux latéraux
- barre de statut si utile

Répartition recommandée

- top : menu + toolbar principale
- left : arbre libre / navigation / bibliothèque / filtres
- center : workspace principal
- right : inspecteur / propriétés / liens / détails
- bottom optionnel : logs, console, timeline légère, sortie technique

Organisation en panneaux

Chaque panneau doit avoir :

- un header clair,
- un titre,
- éventuellement une mini-toolbar locale,
- une zone de contenu bien délimitée.

Types de panneaux attendus

- navigation panel
- tree panel
- asset browser panel
- inspector panel
- preview panel
- graph panel
- property panel
- search/filter panel
- collection panel

Règles

- un panneau = une responsabilité principale
- éviter les panneaux "fourre-tout"

- éviter les headers surchargés
 - regrouper les actions liées localement au panneau
-

Docking et fenêtres flottantes

L'application peut utiliser des panneaux dockables ou flottants ou togglable, mais cela doit rester lisible.

Règles

- les panneaux centraux critiques doivent rester stables
- les fenêtres flottantes servent aux objets secondaires ou focalisés
- ne pas multiplier les popups non ancrées sans nécessité
- la structure par défaut doit rester productive sans reconfiguration manuelle

Usage recommandé

- les conteneurs ou domaines principaux peuvent vivre dans la fenêtre principale
 - les blocs élémentaires ou vues détaillées peuvent s'ouvrir dans une fenêtre flottante ou une vue dédiée
 - les inspecteurs doivent idéalement rester dockés
-

Vue arborescente

L'arborescence est une composante majeure de l'application.

Rôle

Elle sert à :

- organiser librement,
- naviguer rapidement,
- structurer des regroupements,
- afficher des conteneurs et sous-éléments,
- repositionner des objets.

Règles ergonomiques

- support du expand/collapse clair
- sélection visuellement nette
- icônes simples et cohérentes
- renommage direct si pertinent
- drag & drop naturel
- menu contextuel utile
- multi-sélection si nécessaire

Important

L'arbre ne doit pas devenir un mini-tableau illisible.
Limiter les colonnes si elles n'apportent pas une vraie valeur.

Vue centrale de travail

La zone centrale dépend du domaine, mais elle doit toujours rester visuelle et productive.

Types de vues possibles

- grille de miniatures
- liste enrichie
- vue de cartes
- vue graphe
- éditeur dédié
- preview média
- vue interne d'un conteneur

Règle clé

La zone centrale doit montrer les objets d'une manière qui aide à décider, comparer, relier ou éditer.

Priorités

- miniatures visibles pour les images/vidéos
 - intitulés lisibles
 - badges ou métadonnées discrètes
 - espace suffisant pour la sélection et l'ouverture
 - feedback immédiat sur l'objet actif
-

Inspecteur de droite

L'inspecteur doit être stable, lisible et modulaire.

Il peut contenir

- propriétés principales
- description
- tags
- statut
- liens entrants/sortants
- références injectées
- métadonnées techniques
- actions contextuelles

Structure recommandée

L'inspecteur est composé de sections repliables :

- General
- Content
- Metadata
- Links
- References
- Output / Derived data
- Notes / Comments

Règles

- ne pas afficher 50 champs en même temps sans regroupement
- privilégier des sections compactes
- conserver une hiérarchie visuelle forte
- les champs les plus utiles doivent être en haut

Miniatures et cartes

Les thumbnails sont structurants dans ton application.

Objectif

Donner une reconnaissance visuelle immédiate des objets.

Une carte de miniature doit pouvoir afficher

- miniature ou placeholder
- titre
- type (badge avec icon par exemple)
- badges éventuels
- état de sélection
- éventuellement statut ou relation

Règles

- la miniature doit rester prioritaire sur le décor
- pas de surcharge graphique
- titre lisible
- badges petits mais compréhensibles
- état sélectionné très visible
- état vide propre et explicite

La carte doit fonctionner dans plusieurs contextes

- tree enrichi
- grid
- list
- browser
- graph

- résultats de recherche
 - inspecteur synthétique
-

Gestion des états visuels

Chaque vue doit gérer correctement :

- état vide
- absence de données
- chargement
- erreur
- sélection
- survol
- désactivé
- focus

État vide

Un état vide doit toujours expliquer :

- ce qu'il manque,
- ce qu'on peut faire,
- ou pourquoi rien n'apparaît.

Sélection

La sélection doit être immédiatement lisible visuellement, sans ambiguïté.

Hover

Le hover doit aider sans transformer l'interface en sapin de Noël.

Actions et commandes

Toolbar principale

Elle porte les actions globales :

- créer
- importer
- enregistrer
- changer de vue
- filtrer globalement
- lancer certaines actions de haut niveau

Toolbar locale de panneau

Elle porte les actions locales :

- ajouter un élément
- replier/déplier
- filtrer dans le panneau
- changer le mode d'affichage du panneau

Menus contextuels

Ils doivent être utiles, pas redondants.

Ils doivent dépendre de la sélection et exposer les actions logiques.

Densité visuelle

L'application doit être dense mais respirable.

Donc

- éviter les énormes marges inutiles
 - éviter les interfaces compactées au point d'être illisibles
 - garder des espacements cohérents
 - maintenir une hiérarchie claire entre :
 - titre,
 - contenu principal,
 - métadonnées,
 - actions secondaires
-

Icônes

Les icônes doivent :

- rester simples,
- être cohérentes entre elles,
- porter le sens fonctionnel,
- éviter l'illustration excessive.

Les icônes servent à soutenir la lecture, pas à remplacer le texte partout.

Couleurs et accent visuel

Le thème doit rester sobre et professionnel.

Règles

- la couleur d'accent sert à guider l'attention
 - elle ne doit pas être utilisée partout
 - elle marque :
 - la sélection,
-

- l'action principale,
- certains éléments actifs,
- quelques repères de hiérarchie

Ne pas faire

- trop de couleurs concurrentes
 - badges multicolores sans logique
 - contrastes agressifs inutiles
-

Typographie

La typographie doit privilégier :

- lisibilité,
- hiérarchie,
- stabilité.

Règles

- titres de panneaux clairement distincts
 - titres d'objets lisibles
 - métadonnées plus discrètes
 - texte secondaire visuellement secondaire
 - pas d'accumulation de tailles et de graisses différentes
-

Vues par domaine

Même si chaque domaine a sa spécificité, ils doivent partager une base commune.

Caractères / Characters

Doit privilégier :

- organisation en arbre
- visualisation des références et variantes
- preview forte
- accès simple aux images, descriptions, liens

Story

Doit privilégier :

- organisation claire
- lecture narrative ou séquentielle
- accès aux blocs de story et à leurs contenus
- vue centrale orientée structure ou assemblage

Library

Doit privilégier :

- recherche
- navigation
- filtrage
- visualisation rapide des assets

Univers / Location

Doit privilégier :

- regroupement de lieux
- références visuelles
- cohérence des décors
- accès simple aux éléments descriptifs et médias

Liens et relations

Les objets liés doivent être visibles sans rendre l'écran confus.

Règles

- afficher les liens importants de façon synthétique
- distinguer entrants / sortants
- permettre d'ouvrir rapidement l'objet lié
- éviter les listes brutes sans contexte

Dans l'inspecteur, une section dédiée aux liens est préférable à leur dispersion.

Ouverture d'un conteneur

Quand un bloc conteneur est ouvert :

- la vue doit montrer sa structure interne,
- tout en gardant un repère de contexte,
- sans donner l'impression de changer d'application.

Prévoir :

- titre du conteneur actif,
- breadcrumb ou repère clair,
- distinction entre niveau courant et niveau parent.

États de comparaison

Lorsque plusieurs médias ou variantes existent, l'interface doit permettre :

- comparaison rapide,
- reconnaissance visuelle,
- sélection sans confusion.

Éviter les vues qui masquent trop vite les différences entre variantes.

Recherche et filtres

La recherche doit être rapide, visible et non intrusive.

Prévoir si pertinent

- champ de recherche local au panneau
- filtres simples
- tags
- type de bloc
- statut
- vue enregistrée ultérieurement éventuellement

Règle

Les filtres ne doivent pas rendre l'interface incompréhensible.

Toujours pouvoir revenir facilement à l'état normal.

Feedback utilisateur

Toute action visible doit produire un feedback clair si nécessaire :

- sélection
- ouverture
- erreur
- import
- suppression
- renommage
- lien créé
- lien supprimé

Le feedback doit être sobre mais explicite.

Responsive interne

Même sur desktop, l'interface doit rester exploitable lors du redimensionnement.

Donc

- les panneaux doivent tolérer des tailles réduites
- les colonnes trop fragiles sont à éviter

- les sections doivent se replier proprement
 - les widgets doivent avoir des tailles minimales cohérentes
-

Règles de développement pour Codex

Quand un nouvel écran ou widget est développé :

1. identifier si un composant générique existe déjà
 2. le réutiliser ou l'étendre proprement
 3. garder le layout général gauche / centre / droite si pertinent
 4. prévoir les états vides et sélection
 5. connecter l'inspecteur à la sélection
 6. éviter la logique métier dans la vue
 7. produire une démo isolée du widget si possible
-

Résultat attendu global

L'application doit donner l'impression d'un outil :

- sérieux,
- moderne,
- maîtrisé,
- visuel,
- orienté production,
- stable dans son organisation,
- et construit sur une grammaire UI cohérente.

Le but n'est pas de faire une interface spectaculaire.

Le but est de faire une interface durable, claire, professionnelle et extensible.